Progressive Optimization of SGEMM on GPUs: From CUDA Cores to Tensor Cores with Pipelined Execution

Xuanh Cheng 2023202250

***Abstract*—This paper studies the optimization of single-precision matrix multiplication (SGEMM) on NVIDIA GPUs. Starting from CUDA-based implementations, we explore Tensor Core acceleration through the WMMA programming model and further improve performance using a pipelined double-buffering strategy.**

I. Introduction

General matrix multiplication (GEMM) is one of the most fundamental computational kernels in modern computing systems and plays a critical role in both high-performance computing and machine learning workloads.

In this paper, we conduct a progressive study of SGEMM optimization on NVIDIA GPUs. Based on the naive CUDA SGEMM kernel and the shared-memory tiling optimization implemented in previous experiments, we further explore the WMMA programming model under the CUDA framework, where Tensor Cores are used to replace conventional CUDA cores for matrix multiplication. Through further experiments, we observe that introducing shared memory in the WMMA-based implementation leads to performance degradation rather than improvement. Motivated by this observation, we adopt a pipelined execution scheme with double buffering to overlap data movement with Tensor Core computation, thereby improving sustained performance.

II. CUDA-Based SGEMM Implementations

This section presents two SGEMM implementations based on CUDA cores and scalar floating-point operations, which serve as the baseline for the subsequent optimizations and analysis in this paper.

A. Naive CUDA Implementation

In the naive CUDA implementation, each GPU thread is assigned to compute one element of the output matrix C. The thread iterates over the k dimension and accumulates the products of the corresponding elements from matrices A and B. This implementation provides a straightforward parallel mapping and is used as the reference baseline in our experiments.

B. Shared Memory Tiling Optimization

In Experiment 3, a shared memory tiling optimization is introduced. In this approach, sub-tiles of matrices A and B are first loaded from global memory into shared memory by threads within the same thread block. Subsequently, each thread performs computation using data stored in shared memory, enabling reuse of input elements among multiple threads.

By organizing computation in a tiled manner, this optimization significantly improves data locality and reduces global memory accesses compared to the naive implementation. Therefore, the shared memory version serves as a more efficient baseline for comparison with subsequent optimization methods. [1]

III. Tensor Core Acceleration and Pipelined Optimization

The previous CUDA-based implementations rely on scalar computations on CUDA cores. Modern NVIDIA GPUs introduce Tensor Cores, which enable matrix-level multiply-accumulate operations and provide higher peak performance for GEMM workloads.

A. Tensor Cores and WMMA Programming Model

Unlike CUDA cores that implement GEMM by executing many scalar multiply-add operations, Tensor Cores perform matrix multiply-accumulate (MMA) directly in hardware. [2]

Through the WMMA interface, CUDA exposes these hardware MMA operations at warp granularity: one warp (32 threads) cooperatively computes one fixed-size output tile (typically 16×16) of $\mathbf{C}$.

In WMMA, input tiles of $\mathbf{A}$ and $\mathbf{B}$ are loaded into register fragments, and `mma_sync` triggers Tensor Cores to multiply the tiles and accumulate the result into an accumulator fragment. Repeating this step along the K dimension completes the GEMM for the tile, which is then stored back to global memory. For SGEMM, inputs are commonly stored in FP16 while accumulation is performed in FP32. [3]

B. Shared Memory Usage in WMMA-Based SGEMM

Similar to the CUDA-core-based implementation, we attempt to introduce shared memory into the WMMA framework. However, unlike CUDA Core computation, WMMA already achieves data reuse at the warp level through register-resident fragments. Consequently, staging input tiles in shared memory only enables limited reuse across a small number of warps within the same block, while introducing additional synchronization and data movement overhead. As a result, this shared-memory-based approach leads to performance degradation rather than improvement.

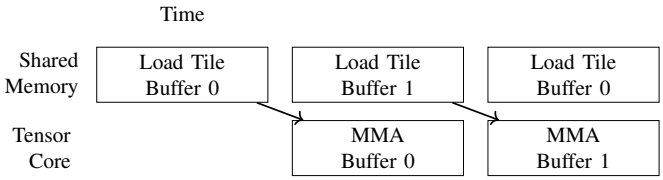

Fig. 1. Pipelined WMMA execution with double buffering.

C. Pipelined Execution with Double Buffering

Although shared memory does not improve WMMA performance through additional data reuse, it can still be used to hide global memory latency. To this end, we adopt a pipelined execution strategy with double buffering.

Specifically, we allocate two shared-memory buffers to stage the input tiles. While the Tensor Core computation consumes tiles from one buffer, the next tiles are prefetched from global memory into the other buffer. After the current MMA computation finishes, the two buffers are swapped, and the pipeline continues.

By overlapping global-to-shared memory transfers with Tensor Core MMA execution, the pipeline reduces the exposed memory latency and improves sustained Tensor Core utilization. As a result, this optimization increases overall SGEMM throughput compared to the WMMA implementation without pipelining.

IV. Experiments

In this section, we experiment with the implemented optimization strategies. The server environment configuration for the experiments is as follows:

- GPU processor for NVIDIA GeForce RTX 3060 Ti

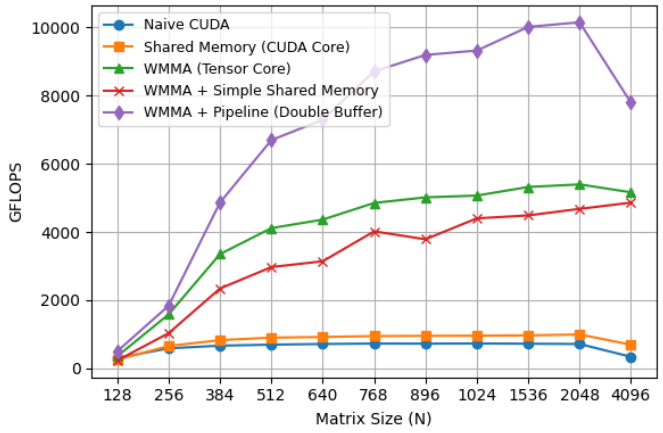

Fig. 2. Processing speeds of different GPU SGEMM implementations.

Figure 2 shows the performance results of different SGEMM implementations across a wide range of matrix sizes.

The shared-memory tiled implementation provides moderate improvement over the naive kernel, for example increasing performance from 585 GFLOPS to 654 GFLOPS at $N = 256$ and from 694 GFLOPS to 899 GFLOPS at $N = 512$. However, its performance saturates below 1 TFLOPS, indicating that CUDA Core–based implementations are constrained by memory bandwidth and instruction throughput.

WMMA-based implementations leveraging Tensor Cores significantly improve performance for medium and large matrix sizes. Throughput increases from 1.58 TFLOPS at $N = 256$ to 5.39 TFLOPS at $N = 2048$, achieving approximately a $7.5\times$ speedup over the naive CUDA kernel. In contrast, introducing shared memory into the WMMA framework does not yield further benefits; for instance, at $N = 1024$, WMMA achieves 5.07 TFLOPS, while WMMA with shared memory reaches only 4.40 TFLOPS, suggesting that additional shared-memory overhead offsets potential gains.

The pipelined WMMA implementation with double buffering delivers the best performance across almost all matrix sizes, reaching a peak throughput of 10.15 TFLOPS at $N = 2048$, about $1.9\times$ higher than the baseline WMMA implementation. By overlapping global memory transfers with Tensor Core computation, this approach effectively hides memory latency. A performance drop at $N = 4096$ indicates possible register pressure or memory bandwidth saturation, leaving room for further optimization.

V. Conclusion and Future Work

This paper explores the optimization of SGEMM on NVIDIA GPUs. Starting from a naive CUDA SGEMM kernel and a shared-memory tiled implementation, we further investigate the use of Tensor Cores through the WMMA programming model, which significantly improves performance for large-scale matrix multiplication.

Experimental results show that introducing shared memory to stage data in the WMMA framework does not provide additional performance benefits and may even lead to performance degradation. By contrast, a pipelined execution strategy with double buffering achieves the highest throughput by overlapping data transfers with Tensor Core computation, demonstrating that latency hiding is more effective than additional data reuse in this context.

Future work will focus on further optimizing data movement for large-scale GEMM, including exploring hybrid memory models and improving the interaction among global memory, shared memory, and Tensor Core execution. In addition, we plan to extend these optimization strategies to other linear algebra kernels and deep learning workloads to broaden their applicability.

References

[1] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, 2008.

[2] NVIDIA Corporation, "Cuda warp matrix multiply accumulate (wmma)," https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#wmma, 2023.

[3] S. Markidis *et al.*, "Nvidia tensor core programmability, performance & precision," in *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2018, pp. 522–531.